

AI Club Coord App

Name: Krishiv Agrawal **Roll No.:** EE25B074

1. Managerial Questionnaire

1.1. Essentials

About Me

Before joining insti, I did not know how to code, but I had a lot of enthu for AI, robotics, and where technology was headed. I was amazed by the amount of innovation happening around me and wanted to be a part of it, even though I did not yet know where to start.

After coming here, I learnt Python, applied to the Quant Guild, and got selected. Later, I joined AI Club as a DC (Deputy Coordinator), and I am currently part of the AI Guild as well. Broadly, I like reading, talking to people, playing sports, and listening to music. I also like environments where I have to move fast, pick up new skills, and iterate quickly.

That is also what pulled me deeper into AI Club. It felt like a place where people were not just discussing ideas, but actually trying to build things with them. That mix of learning, building, and being around people with similar curiosity is something I genuinely enjoy.

Vision for the Club

My vision for AI Club is to educate, ignite interest, and create genuine enthu for AI among freshers. A lot of first-year students are curious about AI, but they usually do not know what to learn first or how to get started. I would like the club to make that first step much easier.

Through GSB sessions, the club should equip freshers with basic ML skills such as:

- coding in Python,
- using Google Colab,
- getting familiar with basic ML terminology,
- participating in beginner-friendly ML hackathons,
- using modern AI tools such as Codex, Claude Code, and similar coding assistants responsibly.

At the same time, AI Club should not stop at beginner sessions. It should also be a place where people can explore more technical and research-oriented directions through club projects. For freshers who want to go deeper, we could have paper-reading sessions, discussions on the current SOTA in different fields, and sessions that expose them to areas like RL, ML systems, CV, speech, NLP, and agents.

Motivation to Become an AI Club Coordinator

Firstly, I want to generate more freshie enthu for AI. A lot of freshers enter insti curious about AI, but that curiosity dies quickly if the first few interactions are either too abstract or too intimidating. Initial hands-on workshops or cool AI app demos can create a much longer-lasting interest.

Secondly, I genuinely want to learn more from the other coordinators and from the work the club does. I am interested in doing research in AI, and I would also like to apply for Project Member at AI Club. Being a coordinator would help me constantly learn, teach, and build around the field.

I also want to improve my soft and interpersonal skills. Explaining technical ideas simply, organizing people, planning sessions, and taking ownership are things I look forward to learning

along the way.

Expected Roles and Responsibilities

As a coordinator, I would have to plan and conduct Epoch sessions, GSB sessions, and other club initiatives to help educate freshers. This would involve choosing topics, making content, arranging logistics, and making sure the sessions are genuinely exciting instead of just informative.

I would also have to recruit DCs, conduct DC sessions, and lead a team of around 7–8 Deputy Coordinators through their mini-project. This would include coming up with the mini-project idea, assigning tasks, helping them when they get stuck, and keeping the team moving till completion.

Taking KTs (Knowledge Transfer Sessions) for other coordinators would also be a part of the role. Finally, I think a coordinator should keep participating in hacks, reading about AI, and learning actively. Reading about the latest research and telling freshers about it would be really cool, and it would also keep the club's content fresh.

Fit With the Club

I think I am a good fit because, by the end of my first year, I have seen AI Club from the inside as one of the DCs and from the outside as someone who attended almost every AI Club event. I have a reasonable sense of what works for first-year students, what loses people, and the different kinds of effort that go into making the club run properly.

I also have the required technical background. I attended DC sessions, completed my DC mini-project, am part of the AI Guild, and have learnt some deep learning on the side. So I can contribute technically, but I also do not see the role as only a technical one. I enjoy talking to people and explaining ideas, which I think matters a lot for conducting sessions and leading DCs.

I also try to keep up with where AI is moving. Recently, the field has been shifting toward agents, coding models, and multimodal systems. That is also the kind of direction I would like AI Club to take sessions on, beyond just standard ML theory.

1.2. The Club

Previous Initiatives and SWOT Analysis

From the AI Club events page, the major initiatives/events listed for the previous tenure are:

- **Knowledge Transfer Sessions (KTS):** a series of ten summer sessions from 16 June to 7 July 2025, covering core ML fundamentals and an introduction to Deep Learning.
- **CFITechSoc: Freshie Roadmap:** a beginner-friendly session for first-year students on 11 August 2025, introducing deep learning from regression to neurons and neural networks.
- **Epoch 1: State Space Models:** a two-day intensive session on 27–28 September 2025, covering sequence modelling from RNNs and Transformers to SSMs, HiPPO, S4, and Mamba.
- **a_i to AI: Where Math Meets ML:** a joint event by Mathematics Club and AI Club on 25 October 2025, connecting probability, optimization, GMMs, EM, and clustering to modern AI.
- **Epoch 2: How Machines Learned to See:** a three-day session from 28–30 January 2026 in collaboration with Jane Street, covering the evolution of computer vision from LeNet/AlexNet to ViTs and ConvNeXt, along with visualizations and adversarial attacks.
- **Bharat Bricks 2026 by Databricks:** an AI-building event/hackathon with a prize pool, mentorship, swag, and a Bengaluru HQ visit.

SWOT 1: CFITechSoc Freshie Roadmap

Strengths	Weaknesses
The session is well-scoped for freshers. It starts from “what is ML?” and moves through supervised/unsupervised/RL, classification vs regression, perceptrons, neural networks, activation functions, forward pass, cost functions, backpropagation, gradient descent, and finally an MNIST neural-net implementation. This gives a beginner a complete first mental model instead of scattered buzzwords.	Because the session covers a lot in about two hours, some parts like backpropagation and gradient descent can feel fast for people seeing ML for the first time. The resource is strong conceptually, but without enough live coding or post-session exercises, freshers may understand the story during the session and still not be able to reproduce it later.
Opportunities	Threats
This can become the official AI Club entry point for the broader first-year audience. It could end with a small take-home notebook: train on MNIST, change hidden-layer size/activation, observe accuracy, and submit a tiny reflection. That would convert the roadmap from a one-time intro into the start of a learning pipeline.	If the first interaction remains mostly explanatory, students may leave with temporary excitement but no habit of building. There is also a risk that students who miss this session do not know how to enter the club’s learning track later.

SWOT 2: Epoch 1 – State Space Models

Strengths	Weaknesses
The event was ambitious and current. The resources build a narrative from word vectors and next-word prediction to RNNs, attention bottlenecks, differential equations, HiPPO, S4, and Mamba. This is exactly the kind of topic that is hard to discover alone as a fresher and shows that AI Club can expose insti to frontier ideas beyond standard ML.	The prerequisite gap is large. Concepts like function spaces, orthogonality, Legendre polynomials, DPLR/NPLR forms, Cauchy kernels, scan operations, and Mamba’s selective SSMs can become overwhelming if the audience does not already have comfort with linear algebra, sequence models, and deep learning basics.
Opportunities	Threats
Epoch can become AI Club’s advanced-track brand: fewer sessions, deeper content, and a final implementation/demo. For SSMs specifically, a small notebook comparing RNN/attention/SSM behaviour on a toy long-context task would make the theory much more concrete. A bridging pre-session could also help people revise word vectors, RNNs, and attention before the main event.	If advanced sessions become too dense, attendance may drop after the initial hype. People may admire the topic but feel that it is “not for them.” This can accidentally create distance between AI Club and the general first-year audience unless there is a clear beginner-to-advanced path.

My main takeaway is that the club already has a good spread: beginner onboarding, technical deep-dives, math-AI crossover events, and hackathon-style building. The next step should be to connect these better into a learning path: Roadmap for entry, Epoch for depth, hackathons for application, and projects for sustained work.

GSB Event Poster Concept

New AI Domains to Explore

As my DC mini-project, I built a plot-hole detector for long stories using a RAG pipeline and a website around it. That project got me interested in the agentic layer. I want to understand how to make agents reliable, where they fail, and how to design workflows that are actually useful instead of just flashy demos.

I am also interested in speech models because I think speech is going to become one of the most natural ways to interact with computers. If voice interfaces become genuinely good, they can change how people use AI tools day to day.

Apart from that, I would like to explore ML systems and multimodal models. ML systems feels important because deployment, inference, latency, and hardware constraints decide whether a model is actually usable. Multimodal models are interesting because they move AI beyond just text and make it interact with images, audio, video, and potentially robotics.

New Club Initiatives

- I would like to conduct hands-on workshops attached to Epoch sessions. I feel each session should have a small build component. For example, after a KNN/PCA session, people could code the algorithm, run it on a toy dataset, visualize it, and modify it. For an LLM session, they could build a small RAG app or agent workflow.
- I did not see many ML hackathons for the GSB from AI Club in my first year, and I would like to improve that. Short events with a clean dataset, or even a simple product problem, would make AI feel more accessible to the broader first-year audience.
- A third idea is to invite professors or researchers for informal research talks. Just exposing freshers to how research problems are framed would be really helpful.
- For freshers who want to explore technical topics more seriously, we could have paper-reading sessions. We can also keep sessions on different fields in AI, such as RL, ML systems, CV, speech, and NLP, along with the current SOTA in each of them.

Initiatives to Continue, Modify, or Pack

- I would not want to pack any initiative completely. I feel a bit of modification would do the work in most cases.
- I would want to modify sessions that have too much theory and too little implementation.
- The issue is not that theory is unimportant. The issue is that for freshers, long theoretical sessions, such as parts of an Epoch series, become hard to focus on and keep up with. I attended all the Epoch sessions, but I did not really grasp all the ideas from them at the time. The content may be useful, but without implementation, it does not stick as well.
- I think the better approach is to use implementation as the hook and theory as the explanation. For example, while taking a session on RNNs, we could first show a visualization, change some parameters, and let people build intuition. Teaching the theory with a coding session later would be much more helpful.

Calendar and Structural Changes

1.3. Publicity and PR

Improving Interest and Enthusiasm Among Insti Junta

- I think the first AI Club interaction for freshers should be less like an introductory lecture and more like a map of the current field.

- We could show the different areas inside AI: computer vision, NLP/LLMs, reinforcement learning, agents, robotics, AI for science, and AI systems. For each area, we should show one strong demo.
- Hands-on workshops can also help a lot, whether it is making websites using AI tools, building small apps, or exploring technical ideas through simple demos.
- For publicity, we can use short demo clips, before-after project reels, “what you will build by the end” posts, and simple beginner roadmaps.

Maintaining Participation Across Multi-Session Events

Participation drops when the first session creates interest but the later sessions do not create momentum. Freshers generally attend the first session because of hype, friends, or pure enthu, but they keep attending only if we spark real interest in them.

So the first sessions are crucial because they set the club’s image. If the first few sessions are abstract or too heavy, people generally lose interest. But if the session gives them a small win, such as building a classifier on a small dataset or making a simple website using AI, they are much more likely to return.

To maintain participation, we could structure the sessions around continuity. Every session should build on top of the previous one. For example, an LLM session track could go like this: build a basic chatbot, add tool use, and then make a deployable app.

1.4. Commitments and PoRs

Other Commitments and Clash Management

I am part of the AI Guild and am applying for Coordinator along with Project Member at AI Club. I do not have any other PoRs planned right now.

If there is a clash between AI Guild work, Project Member work, and coordinator responsibilities, I would prioritize based on deadlines and dependency. Coordinator work would have to be planned in advance because sessions, DC meetings, and mini-project checkpoints affect other people too. I would try to avoid last-minute clashes by keeping a weekly schedule and communicating early whenever a deadline is likely to overlap.

Expected Time Commitment

2. Technical Questionnaire

2.1. Story Time: LLMs and Character-Level Constraints

Why LLMs Struggle With Character-Level Constraints

The problem boils down to tokenization. LLMs do not go through the input query letter by letter. Instead, the text is processed in chunks called tokens, and each token is mapped to a vector. For example, “strawberry” could be a single token or split into something like “straw” and “berry.”

After the tokens are mapped to vectors, the explicit spelling and individual characters are abstracted away. Therefore, when we ask the model to write a story without using the letter “t,” it does not naturally know which of its possible tokens contain a “t” in their string representation. It outputs a sequence of tokens, which are then converted back into text for us to read. This is why word-level constraints are usually easier than exact character-level constraints.

What Happens in Thinking Mode

“Thinking mode” uses a clever trick called test-time compute. A standard model generates text by predicting the next token in sequence, usually with limited intermediate reasoning. In thinking

mode, the model allocates more computation and more time before giving the final answer.

This often looks like Chain-of-Thought reasoning, where the model breaks down the prompt, drafts a response, checks whether it satisfies the constraints, and then self-corrects before answering. So it is not just about scaling the number of parameters. The model is using extra inference-time computation to search, verify, and improve its answer.

Qwen-1.5-1.8B-Chat Challenge Strategy

Evaluating Story Quality

We can evaluate the generated story in multiple ways:

- Use a simple script to check for any occurrence of “t” or “T.”
- Use a word counter to check whether the story satisfies the 300-word constraint.
- Pass the story through a stronger LLM to evaluate fluency, coherence, and whether it reads like an actual detective story.
- Ask a larger LLM or a human evaluator to score narrative quality, creativity, and consistency.

Limits of LLM Capabilities

While LLMs continue to get bigger and improve on benchmarks, I think there is a fundamental limit to only using next-token prediction as the core mechanism. LLMs are extremely powerful statistical pattern matchers, but they do not truly think or create in the same way humans do.

They perform very well on tasks where large amounts of relevant data exist, but they struggle more on tasks like ARC-AGI, where only a few examples are given and the model has to infer the underlying rule. Humans can often identify such rules from very limited data, while LLMs can fail because they are trying to match patterns rather than build a grounded understanding.

However, adding an agentic layer on top of LLMs can make them much more capable. If a model can search, verify, use tools, write code, test hypotheses, and revise its approach, it can perform far beyond a single-pass next-token predictor. So I do not think plain LLMs can do anything, but LLMs combined with tools, agents, and verification loops can become incredibly powerful.

2.2. U-Nets

The Resolution Paradox

The fundamental trade-off is between semantic understanding, i.e. knowing “what” the object is, and spatial localization, i.e. knowing exactly “where” it is.

To classify a pixel accurately, the network needs context from a larger part of the image. Max-pooling helps with this by reducing the spatial size, so deeper layers can effectively see a much larger portion of the original image at once. This aggregation of features into higher-level representations helps the network understand what object or structure is present.

But while doing this, we lose precise positional information. Max-pooling discards exact spatial details during downsampling. This is the core trade-off: we zoom out to understand the whole picture efficiently, but in doing so, we lose some information about exact pixel locations. If we kept the resolution unchanged throughout, the network would need much more computation to build the same amount of global context.

The Memory Bridge

If we delete the skip connections, U-Net effectively becomes a standard encoder-decoder architecture.

In the contracting path, max-pooling creates a highly compressed, semantically rich feature map of the image. The network learns what is present in the image, but it loses a lot of exact spatial

detail. In the expanding path, the network tries to reconstruct a segmentation mask of the original size from this compressed representation.

Skip connections help by passing high-resolution feature maps from the contracting path directly to the expanding path. They give the decoder access to details from the original image, which helps it recover sharper boundaries and better pixel-level localization.

Without skip connections, the network may still identify the general region where the object exists, but the segmentation mask would likely be blurry and approximate. It would struggle with sharp edges, small structures, and complex borders because the decoder would only have access to the compressed bottleneck representation.

The Infinite Canvas

Dense layers require a fixed-size input because they compute something like

$$Y = WX + b.$$

Here, the dimensions of W and b are fixed at training time. Therefore, the input vector X must also have a fixed dimension. If the input image size changes, the flattened vector size changes, and the matrix multiplication no longer works.

Convolutional layers do not have this limitation in the same way. A convolutional layer uses a small kernel, such as a 3×3 filter, and slides it across the image. The same weights are reused at every spatial location. So if the image becomes larger, the filter simply takes more steps across the image.

This gives U-Net flexibility with input dimensions, as long as the dimensions are compatible with the downsampling and upsampling operations. It also makes the model much more memory-efficient than dense layers, because it does not need separate weights for every input pixel.

The Contextual Window

Even though the network uses small 3×3 filters, the effective context grows layer by layer. In the first layer, a 3×3 filter looks at a small patch of the original image and summarizes it. In the next layer, another 3×3 filter looks at a patch of these summaries. Since each summary already represents a region of the original image, the second-layer neuron is effectively seeing a larger patch of the original image.

As the image is repeatedly downsampled in the contracting path, each 3×3 filter corresponds to an increasingly large region in the original input. By the time we reach the bottom of U-Net, a single neuron is looking at a compressed summary of many previous summaries, giving it access to global context.

I do not think massive non-overlapping convolutions are a good replacement. If an important feature like an edge lies on the boundary between two non-overlapping filters, the network may miss it. A massive filter would also require many weights, increasing computation and the risk of overfitting. For example, using non-overlapping 50×50 filters on a 512×512 image would immediately shrink the feature map to around 10×10 , leaving very little spatial structure for deeper layers to work with.

Implementation Task: UNetContractingBlock

The implementation is in `u-net.py`: https://github.com/KrishivAgrawal1/AI_Guild_2026/blob/ai-coord-app/u-net.py. It defines a `UNetContractingBlock` with two unpadded 3×3 convolutions, each followed by ReLU, and prints the tensor shape before the first convolution, after the first convolution, and after the second convolution.

The dummy input has shape $(1, 1, 572, 572)$, corresponding to a batch size of 1, one grayscale channel, and a 572×572 image.

Pixel Drain Explanation and Crop-and-Concat Motivation

For an unpadding convolution, the output size is

$$\text{output size} = \text{input size} - \text{kernel size} + 1.$$

Here, the kernel size is 3, so each convolution reduces both height and width by 2 pixels.

Starting with a 572×572 image:

$$572 - 3 + 1 = 570,$$

so after the first 3×3 convolution, the feature map is 570×570 . After the second convolution:

$$570 - 3 + 1 = 568,$$

so the output becomes 568×568 .

Conceptually, this happens because the kernel cannot be centered on the border pixels unless padding is added. Since the original U-Net uses valid/unpadding convolutions, border pixels are progressively lost. This is the pixel drain.

This creates a problem later during skip connections. Feature maps from the contracting path have to be concatenated with feature maps from the expanding path. But because unpadding convolutions reduce spatial dimensions, the two tensors that need to be concatenated may not have the same height and width.

PyTorch cannot concatenate such tensors directly along the channel dimension unless the spatial dimensions match. So the larger feature map from the contracting path has to be cropped to match the size of the upsampled map. This is why a custom `CropAndConcat` function is needed.

2.3. Humpty Dumpty Had a Great Fall

Mountain Peak Coordinates

The given terrain is

$$f(x_1, x_2, x_3) = \frac{x_1^3}{3} - x_1 + \frac{x_2^5}{5} - x_2 + \frac{x_3^3}{3} - x_3 + 3.$$

To find the candidate peaks/valleys, first compute the gradient:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3} \right) = (x_1^2 - 1, x_2^4 - 1, x_3^2 - 1).$$

Setting this to zero gives

$$x_1 = \pm 1, \quad x_2 = \pm 1, \quad x_3 = \pm 1.$$

Hence there are eight stationary points. The function value at each point is:

(x_1, x_2, x_3)	$f(x_1, x_2, x_3)$
$(1, 1, 1)$	$\frac{13}{11}$
$(1, 1, -1)$	$\frac{5}{37}$
$(1, -1, 1)$	$\frac{15}{19}$
$(1, -1, -1)$	$\frac{5}{11}$
$(-1, 1, 1)$	$\frac{53}{53}$
$(-1, 1, -1)$	$\frac{15}{19}$
$(-1, -1, 1)$	$\frac{5}{77}$
$(-1, -1, -1)$	$\frac{15}{15}$

The largest value is $\frac{77}{15}$, obtained at $(-1, -1, -1)$.

So the mountain peak is at $(-1, -1, -1)$.

Valley or Minima Coordinates

From the same table, the smallest function value among all eight stationary points is $\frac{13}{15}$, obtained at

$$(1, 1, 1).$$

Empirically, among the candidate stationary points, this has the lowest height. So there is a valley, and its coordinates are $(1, 1, 1)$.

Initial Direction for Bounded Descent

Descent Behavior for the Given Direction

Step Sizes That Guarantee Reaching the Minima

Constrained Global Minimum With Walls

Active and Binding Constraints

3. Pedagogy

3.1. Chosen Teaching Topic

Sample direction. Choose the topic you can explain cleanly in 10 minutes. A good flow is: why the topic matters, one visual example, just enough math, one common mistake, and a quick recap.

3.2. Audience Assumptions

3.3. Ten-Minute Explanation Structure

3.4. Core Intuition

3.5. Minimal Mathematical Build-Up

3.6. Example or Demonstration Plan

3.7. Common Confusions and How to Address Them

3.8. Closing Summary

References and LLM Usage Declaration

References

- AI Club, CFI IIT Madras Events Page: <https://aiclubcfi.com/events>
- Google Blog, “Gemini 3.5: frontier intelligence with action.”
- OpenAI Academy, “Introducing GPT-5.3 Instant, GPT-5.4 Thinking, and GPT-5.4 Pro.”
- NVIDIA, “Smart AI Inference at Scale with NVIDIA Blackwell.”

LLM Usage Declaration